

C Programming

A detailed, beginner-friendly study guide

What you will learn

Start with the basics of C, write and run simple programs, understand memory and pointers, and build the foundations needed for technical interviews.

Read one section at a time. Type every example yourself and test small changes.

Table of Contents

No.	Section	Main focus
1	What C Is and Where It Is Used	History, strengths, and uses
2	Setting Up and Running a First Program	Tools, compiling, and running
3	Syntax and Program Structure	Statements, blocks, and comments
4	Variables, Constants, Data Types, and Type Conversion	Storing and changing data
5	Operators and Expressions	Calculations and conditions
6	Input and Output	Reading and displaying values
7	Decision-Making	if, else-if, and switch
8	Loops	for, while, and do-while
9	Functions and Parameter Passing	Reusable blocks and scope
10	Arrays and Strings	Collections of values and text
11	Pointers and Memory Diagrams	Addresses, dereferencing, and safety
12	Structures, Unions, Enums, and Typedef	Organizing related data
13	Dynamic Memory Allocation	malloc, calloc, realloc, and free
14	File Handling	Reading and writing files
15	Preprocessor and Header Files	Macros and declarations
16	Error Handling and	Finding and fixing problems

	Debugging	
17	Common Interview Questions	Short-answer revision
18	Practice Programs with Solutions	Complete beginner programs
19	Exercises with Answers	Try, check, and improve
20	Final Revision Checklist	Last-minute preparation

Learning Objectives

By the end of this guide, you should be able to:

- ☐ Explain what C is, why it is fast, and where it is commonly used.
- ☐ Create, compile, run, and test a small C program.
- ☐ Use variables, constants, data types, operators, and expressions correctly.
- ☐ Read input, display output, and validate simple user choices.
- ☐ Choose between conditions, switch statements, and loops.
- ☐ Write functions and explain how values are passed to them.
- ☐ Work with arrays, strings, structures, and enumerations.
- ☐ Explain pointers with addresses, values, and simple memory diagrams.
- ☐ Allocate and release memory safely when the size is known only at run time.
- ☐ Read and write text files and understand common file errors.
- ☐ Use headers, macros, debugging steps, and good naming habits.
- ☐ Answer common beginner and intermediate C interview questions.

How to Study

1. Read the explanation once without typing.
2. Type the example by hand instead of copying it.
3. Predict the output before running the program.
4. Change one thing, run again, and observe the result.
5. Complete the exercises without looking at the answers first.

Study tip — C becomes easier when you draw the data in memory. For every difficult example, write down the variable, its value, and its address before you run it.

1. What C Is and Where It Is Used

A simple definition

C is a general-purpose programming language. It lets you describe instructions very close to the way a computer works, while still giving you useful tools such as functions, loops, and custom data types.

C was created in the early 1970s and became popular because it is small, fast, portable, and predictable. Many modern languages borrow ideas from C, so learning C improves your understanding of programming in general.

Where C is used

Area	Examples	Why C fits
Operating systems	Core system tools and kernels	Direct control of memory and hardware
Embedded systems	Cars, appliances, sensors, and devices	Small programs can run with limited resources
Compilers and runtimes	Language tools and virtual machines	Speed and low-level control
Networking	Servers, protocols, and utilities	Efficient processing of data
Games and graphics	Engines and performance-heavy parts	Fine control over CPU and memory

Strengths and responsibilities

- **Fast:** C usually compiles to efficient machine instructions.
- **Portable:** the same source can often be compiled on different systems with small changes.
- **Small:** the language has a focused set of core features.
- **Powerful:** pointers allow direct work with memory and hardware interfaces.
- **Less forgiving:** C does not automatically protect you from every memory mistake.

Important idea — C gives you control. Good C programming means using that control carefully: initialize values, check boundaries, release memory, and test assumptions.

2. Setting Up and Running a First Program

What you need

- ☐ A computer with a C compiler, such as GCC or Clang.
- ☐ A text editor or an integrated development environment.
- ☐ A terminal or command prompt.
- ☐ A folder where you can save practice files.

The basic workflow

1. Write source code in a file ending with **.c**.
2. Compile the source file. The compiler checks the language rules and creates an executable program.
3. Run the executable program.
4. Read the output, test different inputs, and fix mistakes.

Command	Meaning
<code>gcc hello.c -o hello</code>	Compile hello.c and name the output hello.
<code>./hello</code>	Run the program on many Unix-like systems.
<code>hello.exe</code>	Run the program on Windows when the output has this name.

First runnable program

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

- **#include:** makes the declaration of printf available.
- **main:** is where the program starts running.

- **printf:** displays text on the screen.
- **return 0:** reports that the program finished successfully.

Common mistake — If the compiler says that a command is not found, the compiler may not be installed or may not be on your system path. If the program runs but shows nothing, check that main contains a printf call and that the output is not hidden by the terminal window.

3. Syntax and Program Structure

What syntax means

Syntax is the set of writing rules for a language. In C, most instructions end with a semicolon. Groups of instructions are placed between braces. C is case-sensitive, so **Value** and **value** are different names.

Part	Purpose
Comment	Explains code to people; it is ignored by the compiler.
Preprocessor line	Requests information before normal compilation begins.
Function	Names a reusable group of instructions.
Statement	Performs one action, usually ending with a semicolon.
Block	Groups statements inside braces.

A small program with structure

```
#include <stdio.h>

int main(void) {
    int score = 80;
    if (score >= 50) {
        printf("Pass\n");
    }
    return 0;
}
```

- **Declaration:** `int score` reserves a name for an integer.
- **Initialization:** `= 80` gives the name its first value.
- **Condition:** the `if` block runs only when the expression is true.
- **Indentation:** spaces do not change what C means, but consistent indentation makes nesting easy to read.

Comments and naming

```
// one-line comment
/* comment that can span
   more than one line */
```

- Use names such as `total_marks` and `student_count`.
- Do not use reserved words such as `int`, `if`, or `return` as variable names.
- Keep one clear purpose per function whenever possible.

4. Variables, Constants, Data Types, and Type Conversion

Variables and constants

A variable is a named storage location whose value can change. A constant is a value that should not change after it is set. Giving a variable its first value is called initialization.

Type	Typical use	Example
int	Whole numbers	<code>int age = 21;</code>
float	Decimal values with moderate precision	<code>float price = 4.50f;</code>
double	Decimal values with more precision	<code>double pi = 3.14159;</code>
char	One character	<code>char grade = 'A';</code>
_Bool	True or false values	<code>_Bool ready = 1;</code>

Constants and limits

```
const int days = 7;
#define MAX_SCORE 100
```

Use `const` for typed values checked by the compiler. Use a macro only when a preprocessor replacement is useful.

Type conversion

Implicit conversion happens automatically. Explicit conversion, also called casting, is requested by writing the target type in parentheses.

```
int a = 5;
int b = 2;
double result = (double)a / b;
printf("%.2f\n", result);
```

- **Integer division:** $5 / 2$ gives 2 when both operands are integers.
- **Decimal division:** converting one operand gives 2.5.
- **Character values:** a char is stored as a small integer value, but use character literals for readable code.

Common mistake — Never use an uninitialized local variable. Its value is indeterminate and may change between runs.

BhavPath-Bhavya Ponduri

BhavPath-Bhavya Ponduri

•

5. Operators and Expressions

What is an expression?

An expression combines values, names, and operators to produce a result. For example, `price * quantity` is an expression. An operator tells C what action to perform.

Group	Operators	Example
Arithmetic	<code>+ - * / %</code>	<code>total % 10</code> gives a remainder.
Comparison	<code>== != > < >= <=</code>	Produces true or false.
Logical	<code>&& !</code>	Combines or reverses conditions.
Assignment	<code>= += -= *= /= %=</code>	Updates a variable.
Increment	<code>++ --</code>	Adds or subtracts one.
Conditional	<code>?:</code>	Chooses one of two values.

Precedence and parentheses

C follows precedence rules. Multiplication normally happens before addition, but parentheses make your intention clear.

```
int answer = 2 + 3 * 4;           // 14
int clear = (2 + 3) * 4;          // 20
int remainder = 17 % 5;           // 2
```

Short-circuit logic

With `&&`, C stops when the left side is false. With `||`, C stops when the left side is true. This can prevent unsafe work.

-

```
if (count > 0 && total / count > 10) {  
    printf("Average is high\n");  
}
```

- **Equality:** use == to compare; use = to assign. **t:** avoid assuming two decimal calculations are exactly equal.

6. Input and Output

Displaying output with printf

The `printf` function writes formatted text. A conversion specifier tells it how to display a value.

Specifier	Value	Example
%d	int	<code>printf("%d", age);</code>
%f	double	<code>printf("%.2f", price);</code>
%c	char	<code>printf("%c", grade);</code>
%s	string	<code>printf("%s", name);</code>

Reading values with scanf

`scanf` needs the address of a variable so it can place the input there. The address operator is `&`.

```
#include <stdio.h>

int main(void) {
    int age;
    printf("Enter your age: ");
    if (scanf("%d", &age) == 1) {
        printf("Next year: %d&bsol;n", age + 1);
    } else {
        printf("Please enter a whole number.&bsol;n");
    }
    return 0;
}
```

Reading text safely

For a line of text, `fgets` is often easier to control than an unrestricted string scan. Always give it the size of the array.

```
char name[40];
fgets(name, sizeof name, stdin);
```

- **Newline:** `\n` moves output to the next line.
- **Input validation:** check the return value of input functions before using the result.
- **Buffer size:** never let input write beyond the array's capacity.

7. Decision-Making with if, else-if, and switch

if and else

An if statement runs a block when its condition is nonzero. An else block runs when the condition is zero.

```
if (temperature > 30) {  
    printf("Hot\n");  
} else {  
    printf("Comfortable\n");  
}
```

else-if chains

Use an else-if chain when there are several exclusive ranges. Conditions are checked from top to bottom, and the first true branch runs.

```
if (mark >= 90) {  
    grade = 'A';  
} else if (mark >= 75) {  
    grade = 'B';  
} else if (mark >= 50) {  
    grade = 'C';  
} else {  
    grade = 'F';  
}
```

switch

Use switch when one expression is compared with several constant cases. A break normally stops execution from continuing into the next case.

```
switch (choice) {  
    case 1:  
        printf("Add&bsol;n");  
        break;  
    case 2:  
        printf("Remove&bsol;n");  
        break;  
    default:  
        printf("Unknown choice&bsol;n");  
}
```

Decision flow

[Start] → [Read value] → {condition?} → Yes: [Action A] → [End]
→ No: [Action B] → [End]

- **Use braces:** braces prevent accidental mistakes when a block grows later.
- **Validate ranges:** reject impossible values such as marks below 0 or above 100.

8. Loops with for, while, and do-while

Why loops are useful

A loop repeats a block while a condition remains true. Every loop needs a clear starting point, a condition, and a progress step so it can eventually stop.

Loop	Best fit	Key question
for	A known count or index range	How many repetitions are needed?
while	Repeat while a condition is true	Should the loop run zero times?
do-while	Run once before checking	Must the body run at least once?

for loop

```
for (int i = 1; i <= 5; i++) {  
    printf("%d\n", i);  
}
```

while and do-while

```
int number = 3;
while (number > 0) {
    printf("%d\\n", number);
    number--;
}

int choice;
do {
    printf("Enter 1 to continue: ");
    scanf("%d", &choice);
} while (choice == 1);
```

break and continue

break leaves the nearest loop. continue skips the rest of the current iteration and starts the next one.

Common mistakes — Forgetting to update the loop variable creates an infinite loop. Using `i <= length` for a zero-based array may access one item too far; usually the condition is `i < length`.

Loop flow

[Initialize] → [Check condition] → No: [End]
→ Yes: [Body] → [Update] → [Check condition]

9. Functions and Parameter Passing

What a function does

A function is a named block that performs one focused task. Functions reduce repetition, make testing easier, and give a program a clear structure.

Part	Example	Meaning
Return type	int	The type of result returned.
Name	square	The function's identifier.
Parameter	int value	Input received by the function.
Return statement	return value * value;	Sends a result back.

Declaration, definition, and call

```
#include <stdio.h>

int square(int value);

int main(void) {
    int result = square(6);
    printf("%d\n", result);
    return 0;
}

int square(int value) {
    return value * value;
}
```

- **Prototype:** tells the compiler about the function before it is used.
- **Definition:** contains the function's actual instructions.
- **Call:** asks the function to run.

How parameters are passed

In ordinary C parameter passing, a function receives a copy of each argument. Changing the copy does not change the original variable.

```
void add_one(int value) {
    value = value + 1;
}

int main(void) {
    int number = 5;
    add_one(number);
    printf("%d\n", number); // 5
}
```

To let a function change the original, pass its address with a pointer. This idea is covered in the pointer section.

Good function habit — Give a function one clear job, choose a descriptive name, keep parameters understandable, and return a useful result or status.

10. Arrays and Strings

Arrays

An array stores several values of the same type next to one another in memory. Array indexes begin at 0. An array with five elements uses indexes 0 through 4.

Index	0	1	2	3
Value	12	7	19	4

```
int scores[4] = {12, 7, 19, 4};
for (int i = 0; i < 4; i++) {
    printf("%d\\n", scores[i]);
}
```

Two-dimensional arrays

Use two indexes for table-like data. The first index identifies a row and the second identifies a column.

```
int grid[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

Strings

A C string is an array of characters ending with a special null character, written as `\\0`. The array must have room for that ending character.

```
char word[] = "cat";
printf("%s has %zu letters\\n", word, strlen(word));
```

Include `<string.h>` for functions such as `strlen`, `strcmp`, and `strcpy`, and use them with valid array sizes.

- **Out of bounds:** accessing an index outside the array is undefined behavior.
- **String comparison:** use `strcmp`, not `==`, to compare string contents.

- **Capacity:** an array's size is fixed after it is created.

11. Pointers Explained Slowly

Start with a variable

When you create a variable, the computer reserves a place in memory. The variable has a value and an address. The address tells where the value is stored.

Variable

name: count

value: 42

Memory idea

address: 1000

stored value: 42

Address and pointer operators

The address operator `&` asks for a variable's address. The dereference operator `*` follows a pointer to the value at that address.

```
int count = 42;
int *pointer = &count;

printf("%d\n", count);
printf("%d\n", *pointer);
```

count → stores **42**
pointer → stores the address of count
*pointer → reaches **42**

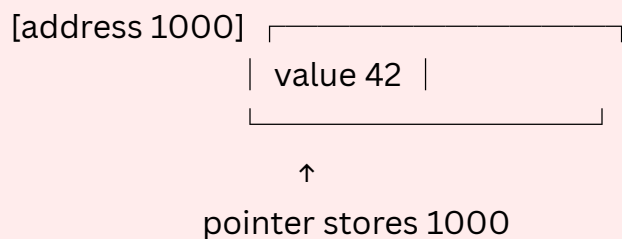
Changing a value through a pointer

```
void double_value(int *number) {  
    *number = *number * 2;  
}  
  
int main(void) {  
    int value = 5;  
    double_value(&value);  
    printf("%d\\n", value); // 10  
}
```

Pointer safety rules

- Initialize a pointer before dereferencing it.
- Use NULL when a pointer intentionally points nowhere.
- Do not use a pointer after the memory it points to has been released.
- Do not read or write beyond an array through a pointer.

Memory diagram



12. Structures, Unions, Enums, and Typedef

Structures

A structure groups fields of different types under one name. It is useful when several values describe one thing, such as a student or a product.

```
struct Student {  
    char name[30];  
    int age;  
    float mark;  
};  
  
struct Student first = {"Mira", 20, 88.5f};  
printf("%s %d&bsol;n", first.name, first.age);
```

Use the dot operator to access a structure variable's field. Use the arrow operator when you have a pointer to a structure.

```
struct Student *p = &first;  
printf("%.1f&bsol;n", p->mark);
```

Unions

A union stores different fields in the same memory location. Only one member should be treated as active at a time. A union can save memory when values are alternatives rather than simultaneous data.

```
union Number {  
    int whole;  
    float decimal;  
};
```

Enums

An enum gives readable names to a set of integer choices.

```
enum Status { PENDING, RUNNING, DONE };  
enum Status job = RUNNING;
```

typedef

typedef creates a shorter or clearer name for an existing type. It does not create a new kind of data.

```
typedef struct {  
    int x;  
    int y;  
} Point;  
  
Point origin = {0, 0};
```

- **Structure:** all fields have separate storage.
- **Union:** members share storage, so one value can overwrite another.
- **Enum:** readable names for integer choices.
- **Typedef:** an alias that improves readability.

13. Dynamic Memory Allocation

Why dynamic memory is needed

An ordinary local array has a size chosen in the source code. Dynamic memory lets a program request space while it is running, for example when the user enters the number of items to store.

Function	Purpose	Initial contents
malloc	Requests a number of bytes.	Uninitialized.
calloc	Requests an array of elements.	All bits are set to zero.
realloc	Changes the size of an existing block.	Preserves available old data.
free	Returns a block to the system.	Pointer must not be used afterward.

Allocating an integer array

```
#include <stdlib.h>

int count = 5;
int *values = malloc((size_t)count * sizeof *values);
if (values == NULL) {
    printf("Allocation failed\n");
} else {
    for (int i = 0; i < count; i++) {
        values[i] = i * 10;
    }
    free(values);
    values = NULL;
}
```

- **Check the result:** allocation can fail, so compare the pointer with NULL.
- **Use sizeof:** calculate the needed number of bytes instead of guessing.

- **Free once:** freeing the same block twice is an error.
- **Free all:** forgetting to release memory creates a memory leak.

Ownership rule — The part of the program that successfully allocates memory should clearly know who will free it. Write the cleanup path before adding more features.

Memory lifecycle

[Request] → [Check] → [Use] → [Release] → [Set pointer to NULL]

14. File Handling

Files and streams

C treats an opened file as a stream of data. A `FILE *` value represents the connection between your program and the file.

Mode	Meaning
r	Open an existing file for reading.
w	Create or overwrite a file for writing.
a	Append new data to the end.
r+	Read and write an existing file.

Writing a text file

```
#include <stdio.h>

int main(void) {
    FILE *file = fopen("notes.txt", "w");
    if (file == NULL) {
        printf("Could not open file.\n");
        return 1;
    }
    fprintf(file, "C practice notes\n");
    fclose(file);
    return 0;
}
```

Reading a file line by line

```
char line[100];
FILE *file = fopen("notes.txt", "r");
if (file != NULL) {
    while (fgets(line, sizeof line, file) != NULL) {
        printf("%s", line);
    }
    fclose(file);
}
```

- **Always check fopen:** the file may not exist or permissions may be missing.
- **Always close:** fclose releases the stream and flushes buffered output.
- **Choose modes carefully:** w can erase an existing file.

Security reminder — Do not build file names from unchecked user input. Validate names and paths in real applications.

15. Preprocessor and Header Files

What happens before compilation

The preprocessor handles lines that begin with `#` before the compiler processes ordinary C statements. It can include declarations, define replacements, and conditionally select parts of a source file.

Directive	Use
<code>#include</code>	Copies declarations from a header into the source file.
<code>#define</code>	Creates a text replacement or macro.
<code>#if</code> , <code>#ifdef</code> , <code>#ifndef</code>	Includes code only when a condition is met.
<code>#endif</code>	Ends a conditional preprocessor region.

Header files

A header usually contains declarations shared by several source files. Standard headers include `<stdio.h>`, `<stdlib.h>`, and `<string.h>`.

```
#ifndef MATH_HELPER_H
#define MATH_HELPER_H

int cube(int value);

#endif
```

The three lines around the declaration form an include guard. It prevents the same header from being included more than once in one compilation.

Macros

```
#define MAX_ITEMS 50
#define SQUARE(x) ((x) * (x))
```

- **Parentheses matter:** wrap macro parameters and the complete expression to reduce surprising results.
- **Prefer const:** for a typed constant that should be checked by the compiler.
- **Keep headers focused:** place related declarations together and avoid unnecessary dependencies.

Simple project idea — Keep declarations in a header, function definitions in a source file, and the program entry point in another source file. This makes larger programs easier to maintain.

16. Error Handling and Debugging

Three kinds of errors

Kind	Example	Usual response
Syntax error	Missing semicolon or brace	Read the compiler message and fix the reported line.
Logic error	Wrong formula or condition	Test expected and actual results step by step.
Run-time error	Invalid memory access or failed file open	Validate inputs, check resources, and inspect state.

A repeatable debugging method

1. Reproduce the problem with the smallest input that still fails.
2. Read the first compiler warning or error, not only the last message.
3. State what you expected and what actually happened.
4. Print or inspect important values near the failure.
5. Change one thing at a time and test again.
6. Add a regression test so the same mistake is less likely to return.

Assertions and return statuses

```
#include <assert.h>

int divide(int total, int count) {
    assert(count != 0);
    return total / count;
}
```

Use assertions for conditions that should always be true during development. For expected user errors, return a status or show a helpful message instead of crashing.

Warning signs to investigate

- A variable is used before it is initialized.

- An array index can become negative or too large.
- A pointer is null, dangling, or never checked.
- Allocated memory is not released on every path.
- A function expects one type but receives another.

Debugging mindset — Do not guess randomly. Reduce the problem, observe the values, form one hypothesis, test it, and keep the smallest fix that solves the real cause.

17. Common Interview Questions

Question	Interview-ready answer
What is the difference between declaration and definition?	A declaration tells the compiler that a name and type exist. A definition creates the function or object and usually allocates storage.
What is a pointer?	A pointer is a variable that stores the address of another object or function.
What is the difference between an array and a pointer?	An array owns a fixed block of elements. A pointer stores an address and can point to different locations, although an array often converts to a pointer in expressions.
What is a null pointer?	A pointer whose value is NULL and therefore is intended to point to no valid object.
What is a memory leak?	Memory that was allocated but can no longer be reached or released by the program.
What is undefined behavior?	The C standard does not define what should happen, so the result may vary and the program cannot be trusted.
Why is sizeof useful?	It reports the size of a type or object in bytes and helps calculate allocation and array capacity correctly.
What does static mean?	Its meaning depends on location: inside a function it preserves a local value between calls; at file scope it limits visibility to that source file.
What does const mean?	The program should not modify the object through that name after initialization.
Why should switch cases use break?	Without break, execution usually continues into the next case, which is called fall-through.

Short answer practice

1. Explain integer division and show how to get a decimal result.
2. Describe how a string ends in C.
3. Explain why `scanf("%d", &number)` uses an address.
4. Compare stack memory and dynamically allocated memory.
5. Explain the difference between `struct` and `union`.

Interview habit — Give a small example, mention one safety concern, and state your assumption. Clear reasoning is often more valuable than a memorized definition.

18. Practice Programs with Solutions

Program 1: Sum from 1 to n

```
#include <stdio.h>

int main(void) {
    int n;
    int sum = 0;
    printf("Enter n: ");
    if (scanf("%d", &n) != 1 || n < 1) {
        printf("Enter a positive integer.\n");
        return 1;
    }
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

The loop visits every number from 1 through n. The condition rejects invalid input before the calculation begins.

Program 2: Find the largest array value

```

#include <stdio.h>

int main(void) {
    int values[] = {12, 5, 27, 9, 18};
    int size = (int)(sizeof values / sizeof values[0]);
    int largest = values[0];

    for (int i = 1; i < size; i++) {
        if (values[i] > largest) {
            largest = values[i];
        }
    }
    printf("Largest = %d\n", largest);
    return 0;
}

```

The first element provides a safe starting value. The loop compares each remaining item with the current largest value.

Program 3: Reverse a string in place

```

#include <stdio.h>
#include <string.h>

int main(void) {
    char text[40];
    printf("Enter text: ");
    if (fgets(text, sizeof text, stdin) == NULL) return 1;
    size_t length = strlen(text);
    if (length > 0 && text[length - 1] == '\n') text[--length] =
'\0';
    for (size_t left = 0, right = length - (length > 0); left < right;
left++, right--) {
        char temp = text[left];
        text[left] = text[right];
        text[right] = temp;
    }
    printf("%s\n", text);
    return 0;
}

```

The two indexes move toward the middle. The temporary character prevents one value from being lost during the swap.

19. Exercises with Answers

Try these first

1. Write a program that prints the even numbers from 2 through 20.
2. Read three integers and print the smallest one.
3. Count how many vowels appear in a line of text.
4. Write a function that swaps two integers using pointers.
5. Store five student marks in an array and calculate their average.
6. Create a structure for a book with a title, author, and price.
7. Write numbers to a file, then read them back and display their sum.
8. Allocate an array for a user-selected number of integers and find its largest value.

Answers and hints

Exercise	Answer or approach
1	Use a for loop with <code>i = 2</code> , condition <code>i <= 20</code> , and update <code>i += 2</code> .
2	Set the first input as the current smallest, then compare the other two values.
3	Loop through the characters and increase a counter when the character is a, e, i, o, or u.
4	Use a temporary value and dereference both pointer parameters to exchange the originals.
5	Use a running integer total, then divide by 5. Cast to double for a decimal average.
6	Declare a structure, create one variable, assign its fields, and print them with the dot operator.
7	Open with write mode, use <code>fprintf</code> , close, reopen with read mode, use <code>fscanf</code> or <code>fgets</code> , and close again.
8	Multiply the count by <code>sizeof</code> the element type, check for NULL, use the block, and free it.

Challenge: trace the output

```
int x = 2;
for (int i = 0; i < 3; i++) {
    x += i;
}
printf("%d\n", x);
```

Answer: 5. The updates are $2 + 0$, then $+1$, then $+2$.

Practice rule — If your answer differs, trace one loop iteration at a time in a small table with columns for i and x .

20. Final Revision Checklist

Core language

- ☐ I can explain what C is and name common uses.
- ☐ I can write a valid main function and return a status.
- ☐ I know when to use semicolons, braces, and comments.
- ☐ I can declare, initialize, and update common data types.
- ☐ I understand integer division, casting, and operator precedence.

Control flow and functions

- ☐ I can choose between if, else-if, and switch.
- ☐ I can write for, while, and do-while loops without accidental infinite loops.
- ☐ I can explain break and continue.
- ☐ I can declare, define, call, and test a function.
- ☐ I understand that ordinary parameters receive copies.

Arrays, memory, and data design

- ☐ I use valid array indexes and leave room for a string's null character.
- ☐ I can explain the difference between an address and the value at that address.
- ☐ I initialize pointers and check for NULL before dereferencing.
- ☐ I can use structures, unions, enums, and typedef appropriately.
- ☐ I check allocation results and free every successful allocation exactly once.

Practical programming

- ☐ I can open, read, write, and close a file safely.
- ☐ I understand headers, include guards, and simple macros.
- ☐ I can separate syntax, logic, and run-time errors.
- ☐ I can explain my assumptions and test edge cases in an interview.

Final readiness test

Without looking at the guide, build one small program that reads input, validates it, uses a function, processes an array, and writes a result to a file. Then explain every variable, loop, condition, and cleanup step aloud.

Keep learning

Read compiler warnings, practice with small programs, revisit pointers often, and prefer clear correct code over clever code. The goal is not to memorize every library function; it is to understand the rules well enough to investigate and build safely.

BhavPath-Bhavya Ponduri

Python Programming Study Guide

From absolute basics to interview-ready foundations

A beginner-friendly, practical learning path

What you will learn

You will learn how to write, run, understand, test, and improve Python programs. The guide uses short explanations, runnable examples, comparison tables, diagrams, practice tasks, and interview preparation.

Estimated study time: 4–6 weeks with regular practice

Level: Beginner to foundational interview readiness

BhavPath-Bhavya Ponduri

Table of Contents

Part	Topic
1	What Python is and where it is used
2	Installing Python and running the first program
3	Variables, names, comments, and indentation
4	Built-in data types and type conversion
5	Operators and expressions
6	Input and output
7	Strings and common string methods
8	if, elif, and else
9	for and while loops
10	Lists, tuples, sets, and dictionaries
11	Functions, parameters, return values, scope, and lambda basics
12	Modules, packages, and imports
13	File handling
14	Exceptions and debugging
15	Object-oriented programming
16	Iterators and generators
17	Useful standard-library concepts
18	Testing and clean code
19	Common interview questions
20	Solved practice programs

21	Exercises with answers
22	Final revision checklist

Suggested order: Read one section, type every example yourself, change one value, predict the result, and then run it.

BhavPath-Bhavya Ponduri

Learning Objectives and Study Plan

Learning objectives

- ☐ Describe what Python is and identify common uses.
- ☐ Install Python, run a script, and use the interactive shell.
- ☐ Use names, variables, values, expressions, and built-in data types.
- ☐ Write decisions, loops, functions, and reusable modules.
- ☐ Work with strings, collections, files, and exceptions.
- ☐ Explain classes, objects, iterators, and generators in simple terms.
- ☐ Read unfamiliar code, find common mistakes, and write basic tests.
- ☐ Solve beginner interview problems using clear Python.

Four-week study plan

Week	Focus	Practice habit
1	Sections 1–8: foundations, types, input, strings, decisions	Type 3–5 small examples each day
2	Sections 9–12: loops, collections, functions, imports	Solve one small problem without copying
3	Sections 13–18: files, errors, classes, generators, testing	Build a small command-line project
4	Sections 19–22: interview practice and revision	Explain solutions aloud and review mistakes

Study rule: Do not only read. Predict the output first, run the example, and write down why your prediction was right or wrong.

1. What Python Is and Where It Is Used

Python is a high-level programming language. High-level means its instructions are written in a form that is easier for people to read than machine instructions. Python emphasizes readable code and lets you solve problems with relatively few lines.

Python is interpreted in the usual beginner workflow: you write instructions, and the Python interpreter reads and executes them. Python also uses dynamic typing, so a name can refer to values of different types at different times.

Area	Typical use
Web development	Build web applications, APIs, and server-side services
Data and science	Clean data, calculate results, and create visual reports
Automation	Rename files, process reports, and repeat office tasks
Artificial intelligence	Experiment with machine learning and language systems
Education	Learn programming with readable examples
Testing	Check whether software behaves as expected

Simple view:

Problem → Python instructions → Interpreter → Result

Remember: Python is useful because it is readable, flexible, and supported by many libraries. It is not the best choice for every situation, but it is an excellent first language.

2. Installing Python and Running the First Program

Step 1 – Install Python. Install a current Python 3 release for your operating system. During installation, enable the option that adds Python to the system path when that option is shown.

Step 2 – Check the installation. Open a terminal and run the Python version command. You should see a Python 3 version number. If the command is not found, reopen the terminal or repair the path setting.

Step 3 – Create a file. Create a file named `first_program.py`. The `.py` ending tells you that the file contains Python instructions.

```
print("Hello, Python!")
name = "Maya"
print("Welcome,", name)
```

Step 4 – Run the file. In the terminal, move to the folder containing the file and run the Python command followed by the filename. The output should contain a greeting and the name.

Line	Meaning
<code>print("Hello, Python!")</code>	Displays text on the screen.
<code>name = "Maya"</code>	Stores text in the name variable.
<code>print("Welcome,", name)</code>	Displays fixed text and the stored value.

Common mistakes: Saving the file with the wrong ending, running the command from the wrong folder, or forgetting quotation marks around text.

3. Variables, Names, Comments, and Indentation

Variable: a name that refers to a value. The assignment operator `=` places a value under a name.

Good name	Why it helps
student_count	Clearly describes a number of students.
total_price	Suggests a calculated amount.
is_ready	Suggests a True or False value.

```
# This is a comment
score = 80
if score >= 50:
    print("Pass")
```

Comments begin with `#` and are ignored when Python runs the program. Indentation is the spaces at the start of a line. Python uses indentation to show which statements belong inside a decision, loop, or function.

Naming rules: names may contain letters, numbers, and underscores; they cannot start with a number; they are case-sensitive; and they must not replace reserved words such as `if` or `class`.

Common mistake: Mixing tabs and spaces or forgetting the colon after a block statement. Use consistent four-space indentation.

4. Built-in Data Types and Type Conversion

Data type describes the kind of value a program is using. Python has many built-in types for common tasks.

Type	Example	Use
int	42	Whole numbers
float	3.14	Numbers with a decimal part
str	"hello"	Text
bool	True	Yes/no conditions
None	None	No value yet or no meaningful result
list	[2, 4, 6]	Ordered, changeable collection

```
age_text = "21"
age = int(age_text)
height = float("1.72")
label = str(404)
ready = bool(1)
```

Conversion means creating a value of another type. Use `int()`, `float()`, `str()`, or `bool()` when the value can be converted safely.

Truthiness: empty strings, zero, empty collections, and `None` behave like `False` in a condition. Most other values behave like `True`.

Common mistake: `int("3.5")` fails because the text is not a whole-number representation. Convert to float first when appropriate.

BhavPath-Bhavya Ponduri

5. Operators and Expressions

An **expression** combines values, names, and operators to produce a result. Operators tell Python what operation to perform.

Group	Operators	Example
Arithmetic	+ - * / // % **	7 // 2 gives 3
Comparison	== != > < >= <=	age >= 18
Logical	and, or, not	is_ready and has_id
Assignment	= += -= *=	count += 1
Membership	in, not in	"a" in "cat"
Identity	is, is not	value is None

```
subtotal = 80
tax = subtotal * 0.10
total = subtotal + tax
print(total)
```

Python follows an order of operations. Parentheses make your intention clear, so write (price * quantity) + delivery instead of relying on memory.

Important: use == to compare values. A single = assigns a value.

6. Input and Output

Output sends information to the screen. The `print()` function can display several values and can format text.

Input reads text typed by the user. The result of `input()` is always a string, even when the user types digits.

```
name = input("Your name: ")
year = int(input("Birth year: "))
age = 2025 - year
print(f"Hello, {name}. Your age is about {age}.")
```

The letter `f` before the string creates an f-string. Text inside braces is replaced by the value of the expression.

Need	Pattern
Show a value	<code>print(value)</code>
Show labels and values	<code>print("Total:", total)</code>
Format a sentence	<code>print(f"Total: {total}")</code>
Read a number	<code>number = float(input("Number: "))</code>

Common mistake: Forgetting to convert input. Compare numbers with numbers, not numeric text.

7. Strings and Common String Methods

A **string** is a sequence of characters. Write strings with single quotes, double quotes, or triple quotes for multi-line text. Strings are immutable, which means a method creates a new string instead of changing the original one.

Method	Example	Result
lower()	"HELLO".lower()	hello
upper()	"hello".upper()	HELLO
strip()	" hi ".strip()	hi
replace()	"red".replace("r", "b")	bed
split()	"a,b".split(",")	["a", "b"]
join()	"-".join(["a", "b"])	a-b
startswith()	"python".startswith("py")	True

```
raw = "  Python is fun  "  
clean = raw.strip().lower()  
words = clean.split()  
print(words)
```

Strings support indexing from zero and slicing. In `word[0:3]`, the start is included and the end is not. Negative indexes count from the end.

Common mistakes: expecting `strip()` to remove characters in the middle, or using an index outside the string length.

8. if, elif, and else

Decision statements choose which instructions to run. Python checks an if condition first, then checks each elif, and finally uses else when no earlier condition is true.

```
score = int(input("Score: "))
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
elif score >= 50:
    grade = "C"
else:
    grade = "Needs practice"
print(grade)
```

Flow:

[condition?] → yes → run block
↓ no
[next condition?] → yes → run block
↓ no
[else block]

Nested decisions place one decision inside another. Keep nesting shallow when possible; complex nesting is harder to read.

Common mistakes: forgetting the colon, using inconsistent indentation, or writing conditions in an order that prevents a later branch from being reached.

9. for and while Loops

A loop repeats instructions. Use a `for` loop when you are visiting items or a known sequence. Use a `while` loop when repetition depends on a condition.

```
for number in range(1, 4):  
    print(number)  
  
count = 3  
while count > 0:  
    print(count)  
    count -= 1
```

`range()` creates a sequence of numbers. The end value is not included. `range(1, 4)` produces 1, 2, and 3.

Control word	Purpose
<code>break</code>	Stops the loop immediately
<code>continue</code>	Skips to the next iteration
<code>pass</code>	Does nothing; useful as a temporary placeholder

Common mistake: A while loop must eventually make its condition false. If the condition never changes, the loop may run forever.

10. Lists, Tuples, Sets, and Dictionaries

Collections hold multiple values. Choose one based on whether order, changeability, uniqueness, or key-based lookup matters.

Collection	Example	Best for
list	[10, 20, 30]	Ordered values that may change
tuple	(10, 20, 30)	Ordered values that should not change
set	{10, 20, 30}	Unique values and membership checks
dictionary	{"name": "Asha"}	Values found by keys

```
scores = [72, 88, 91]
scores.append(95)
print(scores[0])

person = {"name": "Asha", "age": 20}
print(person["name"])

unique = set([1, 1, 2, 3])
```

Mutability: lists and dictionaries can change after creation. Tuples and strings cannot be changed in place. A set does not promise a useful display order.

Common mistakes: using a missing dictionary key, confusing a set with a dictionary, or changing a collection while looping over it.

11. Functions, Parameters, Return Values, Scope, and Lambda Basics

A **function** is a named block of reusable instructions. Define it with `def`. Parameters receive input, and `return` sends a result back to the caller.

```
def area(width, height=1):  
    result = width * height  
    return result  
  
box_area = area(5, 2)  
print(box_area)
```

Parameter types: positional arguments match by order, keyword arguments match by name, and default values are used when an argument is not supplied.

Scope describes where a name can be used. A local variable belongs to a function. A global variable is defined outside functions. Prefer passing values into functions instead of changing global data.

Lambda is a small anonymous function. It is useful for a short expression, such as `double = lambda number: number * 2`. Use a normal function when the logic needs explanation or several lines.

Good habit: Give each function one clear job, use descriptive parameter names, and return data instead of printing from every function.

12. Modules, Packages, and Imports

A **module** is a Python file containing reusable names. A **package** is a folder that groups related modules. An **import** makes names from another module available in your program.

```
import math
print(math.sqrt(81))

from random import randint
print(randint(1, 6))

import statistics as stats
print(stats.mean([2, 4, 6]))
```

Import styles:

- **import module** keeps the module name visible and avoids many naming conflicts.
- **from module import name** lets you use one selected name directly.
- **import module as short_name** creates a readable alias.

When running your own file, keep related files in a clear folder and avoid naming your file after a standard module, such as `random.py`.

Common mistake: Importing a name that does not exist, creating circular imports, or forgetting that an import runs the imported module's top-level instructions.

13. File Handling

File handling lets a program save information and read it later. Use `open()` with a mode and a `with` statement. The `with` statement closes the file automatically, even when an error occurs.

Mode	Meaning
r	Read an existing file
w	Write from the beginning; replaces old content
a	Append to the end
rb or wb	Read or write binary data

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Practice a little every day.&#92;n")

with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()
print(contents)
```

Use `read()` for all text, `readline()` for one line, or loop over the file for memory-friendly processing.

Common mistake: Using write mode when you meant append mode. Write mode can remove the old contents.

14. Exceptions and Debugging

An **exception** is a problem reported while a program runs. Examples include invalid input, a missing file, a missing dictionary key, and dividing by zero. Handle only errors you understand and can respond to usefully.

```
try:
    number = int(input("Number: "))
    result = 100 / number
except ValueError:
    print("Please enter a whole number.")
except ZeroDivisionError:
    print("The number cannot be zero.")
else:
    print(result)
finally:
    print("Finished.")
```

Debugging process:

1. Read the complete error message and locate the line named in the traceback.
2. Reproduce the problem with the smallest input that still fails.
3. Print or inspect important values before the failing line.
4. Fix one idea at a time and run the program again.
5. Add a test so the same bug is less likely to return.

Do not hide every error: A broad exception handler can make a broken program appear successful. Catch specific exceptions whenever possible.

15. Object-Oriented Programming with Classes and Objects

Object-oriented programming groups data and behavior together. A **class** is a design or blueprint. An **object** is a particular value created from that class. Attributes store data, and methods define behavior.

```
class Counter:
    def __init__(self, start=0):
        self.value = start

    def increment(self):
        self.value += 1

counter = Counter(10)
counter.increment()
print(counter.value)
```

Important lines: `__init__` runs when an object is created. `self` refers to the current object. `self.value` stores data on that object.

Idea	Plain-English meaning
Encapsulation	Keep related data and operations together.
Inheritance	Create a new class based on an existing class.
Polymorphism	Use the same operation with different object types.

Common mistake: Forgetting `self` in a method or using a class when a simple function would be clearer. Use the simplest design that solves the problem.

16. Iterators and Generators

Explained Simply

An **iterable** is something you can loop over, such as a list or string. An **iterator** remembers its current position and provides the next value when requested.

Generator: a simple way to create an iterator. A function containing `yield` produces one value at a time and pauses until the next value is requested.

```
def count_up_to(limit):  
    number = 1  
    while number <= limit:  
        yield number  
        number += 1  
  
for value in count_up_to(3):  
    print(value)
```

Memory idea:

List: [1, 2, 3, 4] → all values are ready

Generator → 1 → pause → 2 → pause → 3 → pause → stop

Generators are useful when the full result would be large. They can reduce memory use because they do not create every value at once.

Simple rule: Use a list when you need to revisit all values. Use a generator when you want to process values one at a time.

17. Useful Standard-Library Concepts

The standard library is the collection of modules that comes with Python. Learn a few dependable tools before searching for an outside package.

Module	Useful idea	Example use
math	Mathematical functions	sqrt, ceil, floor
random	Random choices and numbers	choice, randint
datetime	Dates and times	Get today's date
pathlib	Work with file paths	Check whether a file exists
json	Save structured data as text	Load settings
collections	Special collection helpers	Counter, deque
re	Pattern matching	Find structured text

```
from pathlib import Path
folder = Path(".")
for item in folder.iterdir():
    print(item.name)
```

Use documentation and small experiments to learn unfamiliar modules. Read function names, required arguments, returned values, and raised exceptions before using a tool in a larger program.

18. Introduction to Testing and Clean Code

Testing means checking that code produces the expected result. A small test usually gives an input, calls the code, and compares the result with the expected answer.

```
def add(a, b):  
    return a + b  
  
def test_add():  
    assert add(2, 3) == 5  
    assert add(-1, 1) == 0
```

Test boundaries: test ordinary values, zero, negative values, empty input, the smallest valid input, and invalid input where appropriate.

Clean-code habit	Why it helps
Use meaningful names	Readers understand purpose without guessing.
Keep functions short	Small behavior is easier to test.
Avoid repetition	One fix can improve every use.
Write simple conditions	Logic is easier to verify.
Comment the reason	Useful comments explain decisions, not obvious syntax.

Quality loop: Write a small piece → run it → test it → read it again → simplify it.

19. Common Python Interview Questions

Question	Beginner-friendly answer
What is the difference between a list and a tuple?	A list is changeable; a tuple is generally used for an ordered group that should not change.
What is the difference between == and is?	== compares values. is checks whether two names refer to the same object.
Why is input often converted?	input() returns text, so convert it when arithmetic or another type is needed.
What does None mean?	It represents the absence of a value or an intentionally empty result.
What is a mutable object?	An object that can be changed after creation, such as a list or dictionary.
What is a dictionary?	A collection that maps unique keys to values.
What is a generator?	An iterator-producing function that gives values one at a time with yield.
Why use exceptions?	To respond to expected runtime problems without crashing unexpectedly.
What is a class?	A blueprint for creating objects that combine data and behavior.

Interview approach: define the idea, give a tiny example, mention one trade-off, and explain how you would test it. If unsure, say what you would verify rather than inventing an answer.

20. Solved Practice Programs

Program 1 — Check whether a number is even

```
number = int(input("Enter a number: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Why it works: the remainder operator gives zero when the number divides evenly by two.

Program 2 — Find the largest value

```
values = [12, 5, 27, 9]
largest = values[0]
for value in values[1:]:
    if value > largest:
        largest = value
print(largest)
```

Why it works: begin with the first value, compare each remaining value, and replace the stored answer when a larger value appears.

Program 3 — Count vowels

```
text = input("Enter text: ").lower()
count = 0
for character in text:
    if character in "aeiou":
        count += 1
print("Vowels:", count)
```

Practice change: modify this program to count consonants, digits, or each individual vowel.

BhavPath-Bhavya Ponduri

20. Solved Practice Programs — Continued

Program 4 — Create a simple multiplication table

```
number = int(input("Number: "))
for multiplier in range(1, 11):
    result = number * multiplier
    print(f"{number} x {multiplier} = {result}")
```

Program 5 — Word frequency

```
sentence = "red blue red green blue red"
counts = {}
for word in sentence.split():
    counts[word] = counts.get(word, 0) + 1
print(counts)
```

Why it works: `get()` supplies zero for a word that has not appeared. Each appearance then increases its count.

Program 6 — Safe division function

```
def safe_divide(first, second):
    if second == 0:
        return None
    return first / second

answer = safe_divide(10, 2)
print(answer)
```

Design lesson: the function returns data and lets the caller decide how to display a missing result.

BhavPath-Bhavya Ponduri

21. Exercises with Answers

Exercises

1. Write a program that prints numbers from 1 to 20 but skips multiples of 3.
2. Ask for a sentence and print the number of words.
3. Create a list of numbers and calculate its average without using a statistics module.
4. Build a dictionary that counts each character in a word.
5. Write a function that returns True when a word is a palindrome.
6. Read a text file and count its lines.
7. Create a class named Book with a title and an author.
8. Write a generator that produces the first five square numbers.

Answers and hints

No.	Answer idea
1	Loop through <code>range(1, 21)</code> , use <code>continue</code> when <code>number % 3 == 0</code> .
2	Use <code>sentence.split()</code> and apply <code>len()</code> to the resulting list.
3	Keep a total and count while looping, then divide total by count.
4	Use <code>counts.get(character, 0) + 1</code> inside a loop.
5	Compare <code>word</code> with <code>word[::-1]</code> .
6	Use <code>with open(..., "r", encoding="utf-8")</code> and count lines in a loop.
7	Use <code>__init__</code> to assign <code>self.title</code> and <code>self.author</code> .
8	Use a loop from 1 to 5 and <code>yield number * number</code> .

Self-check: For every answer, test an ordinary input, an empty input where valid, and a boundary case.

BhavPath-Bhavya Ponduri

21. Exercises with Answers — Worked Examples

Palindrome function

```
def is_palindrome(word):  
    cleaned = word.lower().strip()  
    return cleaned == cleaned[::-1]  
  
print(is_palindrome("Level"))  
print(is_palindrome("Python"))
```

The slice `[::-1]` reads the string from the end to the beginning. Comparing the cleaned word with its reverse answers the question directly.

Character counter

```
def character_counts(text):  
    counts = {}  
    for character in text:  
        counts[character] = counts.get(character, 0) + 1  
    return counts
```

Improve it: decide whether spaces should be counted. If not, skip them with a condition before updating the dictionary.

Square-number generator

```
def squares(limit):  
    for number in range(1, limit + 1):  
        yield number * number  
  
for square in squares(5):  
    print(square)
```

Explain it aloud: the function does not build a complete list. It pauses after each square and continues when the loop asks for another value.

BhavPath-Bhavya Ponduri

22. Final Python Revision Checklist

Foundations

- ☐ I can explain what Python is and name several areas where it is used.
- ☐ I can install Python, create a file, run it, and use the interactive shell.
- ☐ I understand names, assignment, comments, indentation, and naming rules.

Core language

- ☐ I can identify strings, integers, floats, booleans, lists, dictionaries, tuples, sets, and None.
- ☐ I can convert input to the correct type and explain truthiness.
- ☐ I can use arithmetic, comparison, logical, membership, and assignment operators.
- ☐ I can write clear input and output with f-strings.
- ☐ I can write if, elif, else, for, while, break, and continue correctly.

Reusable programs

- ☐ I can create functions with parameters, defaults, return values, and a clear purpose.
- ☐ I can explain local scope and the basic use of lambda expressions.
- ☐ I can import a module and explain the difference between a module and a package.
- ☐ I can read and write text files safely with a with statement.

Professional foundations

- ☐ I can catch specific exceptions and use a traceback to guide debugging.
- ☐ I can explain classes, objects, attributes, methods, and self.
- ☐ I can explain iterables, iterators, generators, and yield simply.
- ☐ I know a few useful standard-library modules and when to use them.
- ☐ I can write basic assertions and choose meaningful test cases.
- ☐ I can solve small problems and explain my choices in an interview.

Final challenge: Build one small project that accepts input, stores data in a collection, uses functions, handles an error, saves a result to a file, and includes at least five tests.

Keep learning: revisit the sections where you made mistakes, rewrite examples without looking, and practice explaining each solution in plain English.

